SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA1011	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	HARIHARAN.R	**Reg. No.:**	192524003
Date:	23.10.2026	**Duration:**	60 minutes

Code of Conduct

I _____HARIHARAN.R(192524003)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____HARIHARAN.R_____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and	Diagram is incomplete, unclear, or

	major components, interfaces, and interactions accurately represented.		lacks sufficient detail.	technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

Food Delivery Application: Design an architecture for a food delivery system involving customers,
restaurants, delivery partners, order management, and payment processing.
ANSWER::

FOOD DELIVERY APPLICATION – SYSTEM ARCHITECTURE DESIGN

1. AIM

To design a scalable, secure, reliable, and efficient architecture for a Food Delivery Application that connects customers, restaurants, delivery partners, order management, and payment processing.

2. INTRODUCTION

A Food Delivery Application is an online platform that enables customers to order food from nearby restaurants through a website or mobile application. It allows customers to browse menus, add food items to their cart, make payments, and track their orders in real time.

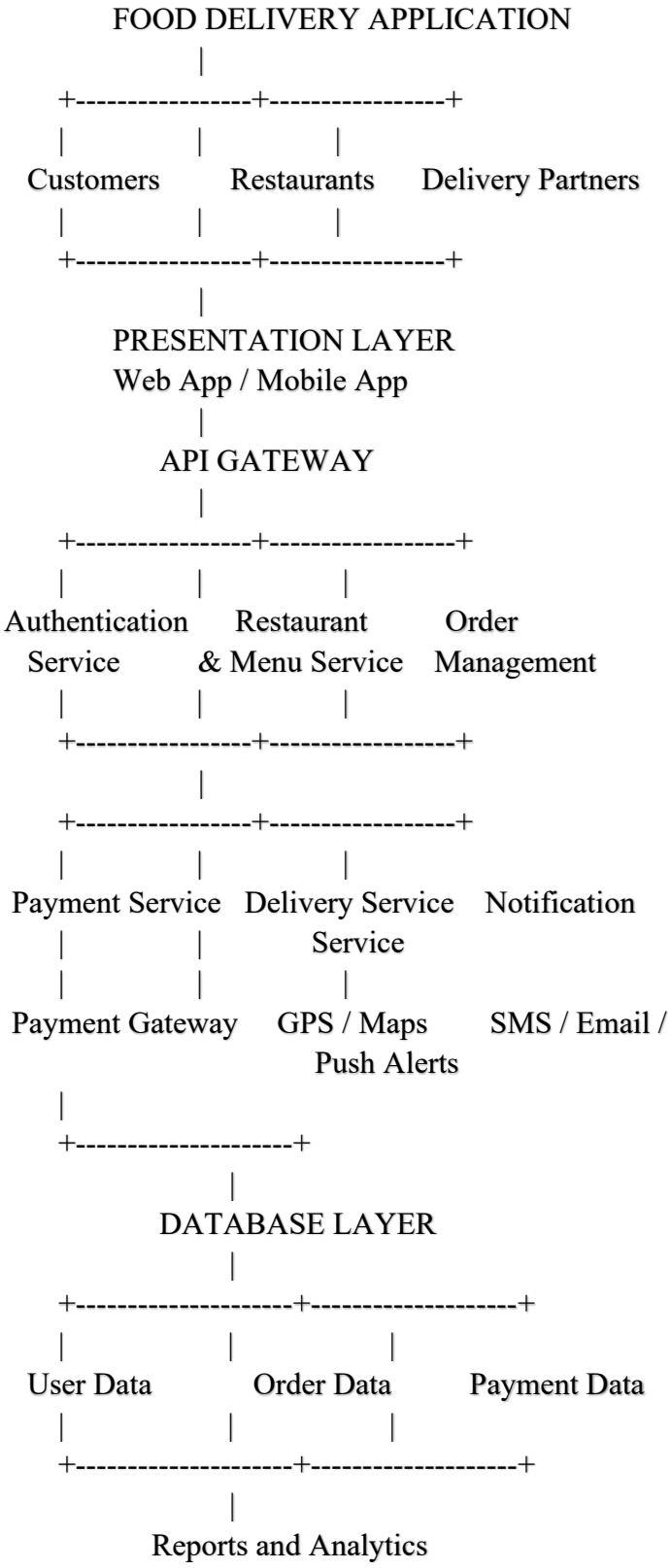
The system connects three major users: customers, restaurant owners, and delivery partners. It also includes supporting services such as order management, payment processing, notifications, location tracking, and database management.

The proposed system uses a layered architecture with modular services. Each service performs a specific responsibility, making the application easier to maintain, update, test, and scale.

3. OBJECTIVES

1. To provide a user-friendly platform for online food ordering.
2. To enable restaurants to manage menus, prices, and incoming orders.
3. To assign delivery partners and track deliveries efficiently.
4. To provide secure online payment processing.
5. To send real-time order status notifications.
6. To maintain accurate customer, order, and payment records.
7. To improve system performance, availability, and reliability.
8. To reduce manual work and improve the overall food delivery experience.

4. SYSTEM ARCHITECTURE DIAGRAM



5. ARCHITECTURAL STYLE

The proposed system uses a layered architecture combined with modular services.

5.1 Presentation Layer

This layer provides the user interface through a mobile application or website.
Functions:

- Customer registration and login.
- Restaurant browsing and food selection.
- Shopping cart and checkout.
- Restaurant order management screens.
- Delivery tracking and order status display.

5.2 Application / Business Logic Layer

This layer processes the main business operations.
Functions:

- User authentication and authorization.
- Menu and restaurant management.
- Order validation and processing.
- Payment verification.
- Delivery partner assignment.
- Notifications and status updates.

5.3 Data Layer

This layer stores and retrieves application information.
Functions:

- Stores user profiles and addresses.
- Maintains restaurant menus and food prices.
- Stores orders and transaction records.
- Maintains delivery partner information.
- Supports database backups and recovery.

5.4 External Integration Layer

This layer connects the application with third-party services.
Functions:

- Payment gateway integration.
- GPS and map services.
- SMS and email providers.
- Push notification services.

6. MAJOR SYSTEM COMPONENTS

6.1 Customer Module

- Allows customers to register, log in, and manage their profiles.
- Displays nearby restaurants based on location.
- Provides menus, food descriptions, prices, and ratings.
- Allows customers to add, remove, and update items in the cart.
- Supports online ordering and order cancellation under defined conditions.
- Enables customers to track delivery status.
- Provides order history, feedback, and ratings.

6.2 Restaurant Management Module

- Allows restaurant owners to register and manage their restaurant profiles.
- Enables adding, updating, and deleting menu items.
- Maintains food prices and item availability.
- Receives incoming orders from customers.
- Allows restaurants to accept or reject orders.
- Updates food preparation and ready-for-pickup status.
- Provides daily order summaries and sales reports.

6.3 Delivery Partner Module

- Allows delivery partners to log in and manage their profiles.
- Displays available delivery requests.
- Assigns orders based on availability, location, and delivery distance.
- Provides pickup and customer delivery addresses.
- Uses GPS to support location tracking.
- Updates order status after pickup and delivery.
- Records completed deliveries and delivery history.

6.4 Order Management Service

- Receives orders from customers.
- Validates selected food items and their availability.
- Calculates the total cost, including delivery charges, taxes, and discounts where applicable.
- Sends orders to the selected restaurant.
- Maintains order status and timestamps.
- Handles eligible cancellations and order failures.

6.5 Payment Processing Service

- Supports available online payment methods.
- Connects to a secure payment gateway.
- Verifies payment success or failure.
- Maintains transaction references and payment status.
- Supports eligible refunds.
- Prevents duplicate payment processing through idempotency controls.
- Does not store sensitive card details unless the system is specifically designed and authorized to do so.

6.6 Notification Service

- Sends order confirmation messages.
- Notifies restaurants when new orders arrive.
- Alerts customers about payment success or failure.
- Sends updates when food is being prepared or picked up.
- Provides delivery status notifications.
- Sends alerts through push notifications, SMS, or email.

6.7 Location and Tracking Service

- Identifies the delivery partner's current location when location sharing is enabled.
- Displays the delivery partner on a map.
- Helps estimate delivery time.
- Supports route planning using map services.
- Updates customers when the delivery partner approaches the destination.

6.8 Review and Feedback Module

- Allows customers to rate restaurants and food items.
- Collects delivery experience feedback.
- Helps restaurants identify service issues.
- Stores reviews for future reference.
- Supports moderation of inappropriate feedback.

6.9 Admin Module

- Allows administrators to manage customer and restaurant accounts.
- Monitors orders, payments, and delivery activities.
- Handles complaints and disputed transactions.
- Reviews suspicious activity.

- Generates operational and sales reports.
- Manages service availability and system configuration.

7. DATABASE DESIGN

The database stores the information required for the application's operation.

7.1 Users Table

- User_ID
- Name
- Email
- Phone_Number
- Password_Hash
- Address
- Role

7.2 Restaurants Table

- Restaurant_ID
- Restaurant_Name
- Location
- Contact_Number
- Opening_Hours
- Restaurant_Status

7.3 Menu Table

- Item_ID
- Restaurant_ID
- Item_Name
- Description
- Price
- Availability_Status

7.4 Orders Table

- Order_ID
- Customer_ID
- Restaurant_ID
- Order_Date
- Total_Amount

- Order_Status
- Delivery_Address

7.5 Order Items Table
- Order_Item_ID
- Order_ID
- Item_ID
- Quantity
- Unit_Price
- Subtotal

7.6 Payments Table
- Payment_ID
- Order_ID
- Transaction_Reference
- Payment_Method
- Amount
- Payment_Status

7.7 Delivery Table
- Delivery_ID
- Order_ID
- Delivery_Partner_ID
- Pickup_Time
- Delivery_Time
- Delivery_Status

Database relationships:
- One customer can place multiple orders.
- One restaurant can offer multiple menu items.
- One order can contain multiple food items.
- One order can have associated payment records, including retries or refunds.
- Each delivery assignment is linked to an order and a delivery partner.

8. WORKING PROCEDURE
1. The customer opens the application and logs in.
2. The customer searches for a restaurant and views its menu.

3. The customer selects food items and adds them to the cart.
4. The system checks item availability and calculates the total amount.
5. The customer confirms the delivery address and selects a payment method.
6. The Payment Service requests payment authorization from the payment gateway.
7. The system records the payment result and creates or confirms the order according to the defined order-processing workflow.
8. The restaurant receives the order and accepts it for preparation.
9. The restaurant updates the food preparation status.
10. The system finds an available delivery partner and assigns the delivery.
11. The delivery partner collects the prepared food.
12. The customer tracks the delivery using the application.
13. The delivery partner hands over the order and marks it as delivered.
14. The system updates the final order status and sends a completion notification.
15. The customer can submit a rating and feedback.

9. ORDER STATUS MANAGEMENT

The system maintains different order states to ensure proper tracking.

- Placed: The customer has submitted the order.
- Payment Pending: Payment processing is in progress.
- Confirmed: The order has been accepted according to the business rules.
- Preparing: The restaurant is preparing the food.
- Ready for Pickup: The food is ready for collection.
- Picked Up: The delivery partner has collected the order.
- Out for Delivery: The order is travelling to the customer.
- Delivered: The customer has received the food.
- Cancelled: The order has been cancelled according to the cancellation policy.
- Failed: The order could not be completed.

The system should allow only valid status transitions and record timestamps for important events.

10. SECURITY MECHANISMS

- Use HTTPS to protect data transmitted between clients and servers.
- Store passwords using secure password-hashing algorithms.
- Apply role-based access control for customers, restaurants, delivery partners, and administrators.
- Validate user input to reduce common application vulnerabilities.
- Protect payment operations through a trusted payment gateway.

- Store secrets and API keys securely.
- Apply rate limiting to reduce abuse and automated attacks.
- Maintain audit logs for important administrative and financial actions.
- Protect customer addresses and location data through access restrictions.
- Use secure session management and appropriate account recovery procedures.

11. ERROR HANDLING AND RECOVERY

- If payment fails, display the failure reason where available and allow a safe retry.
- If a restaurant rejects an order, notify the customer and apply the refund policy where necessary.
- If a delivery partner becomes unavailable, reassign the delivery.
- If a network error occurs, retry eligible requests safely.
- Prevent duplicate orders and payments using unique request identifiers and idempotency controls.
- Store important order changes so that failed operations can be investigated.
- Back up essential data and test database recovery procedures.
- Provide clear error messages without exposing confidential system information.

12. NON-FUNCTIONAL REQUIREMENTS

12.1 Performance

The application should load restaurant menus quickly and process orders with minimal delay.

12.2 Scalability

The architecture should support increasing numbers of customers, restaurants, and delivery requests.

12.3 Availability

The system should minimize downtime and recover from service failures.

12.4 Reliability

Order and payment information must remain accurate and consistent.

12.5 Security

The system must protect customer data, account credentials, and financial transactions.

12.6 Maintainability

Individual modules should be easy to test, update, and maintain.

12.7 Usability

The interface should be simple, clear, accessible, and easy to navigate.

12.8 Interoperability

The application should communicate correctly with payment gateways, map services, and notification providers.

12.9 Consistency

The system should maintain correct order states and payment records even when multiple operations occur at the same time.

12.10 Observability

Logs, metrics, and alerts should help administrators identify errors and monitor system performance.

13. TESTING STRATEGY

- Unit Testing: Tests individual functions such as price calculation and order validation.
- Integration Testing: Checks communication between order, payment, restaurant, and delivery services.
- System Testing: Verifies the complete food ordering and delivery workflow.
- Security Testing: Checks authentication, authorization, input validation, and data protection.
- Performance Testing: Measures response times under normal and heavy workloads.
- Usability Testing: Evaluates whether customers and delivery partners can use the application easily.
- Recovery Testing: Checks whether the application recovers correctly from network, database, or service failures.
- Acceptance Testing: Confirms that the application meets the requirements of customers, restaurants, and administrators.

14. ADVANTAGES

1. Provides convenient online food ordering.
2. Reduces manual order-processing work.
3. Improves communication between customers and restaurants.
4. Supports transparent order and delivery tracking.
5. Provides secure payment processing.
6. Improves delivery assignment and operational efficiency.
7. Helps restaurants monitor orders and sales.
8. Supports business growth through scalable architecture.
9. Improves customer satisfaction through timely notifications.

15. FUTURE ENHANCEMENTS

1. AI-based restaurant and food recommendations.
2. Machine-learning models for delivery-time prediction.
3. Dynamic delivery partner assignment based on traffic and distance.
4. Voice-based food ordering.
5. Chatbot support for customer queries.
6. Demand forecasting for restaurants.
7. Loyalty programs and personalized discounts.
8. Fraud detection for suspicious transactions.
9. Multi-language support.
10. Environmentally friendly delivery route optimization.

16. CONCLUSION

The proposed Food Delivery Application architecture provides an organized solution for connecting customers, restaurants, and delivery partners. The presentation layer provides a user-friendly interface, the business logic layer manages orders and transactions, and the data layer stores essential information.

By integrating order management, payment processing, delivery tracking, notifications, security, and error recovery, the system improves efficiency and reliability. The modular architecture also supports future enhancements and increasing user demand. Therefore, the proposed design provides a strong foundation for developing a modern, secure, scalable, and user-friendly food delivery platform.